

api-oficina-mecanica × ENG-TECH-CHALLENGE.pdf

Revisão do que está **de fato implementado** contra as regras de negócio do enunciado — não do que está planejado. Cada item traz o veredito, a evidência no código e, quando há lacuna, uma correção concreta.

Branch **feature/os-status-change** · 485 testes · 97% cobertura ·

Revisor: **auditoria automatizada**

9

CONFORME

2

PARCIAL

4

LACUNA

2

CRÍTICO

A

Criação da Ordem de Serviço

"Identificação do cliente por CPF/CNPJ; cadastro do veículo; inclusão de serviços; possibilidade de incluir peças/insumos; orçamento gerado automaticamente; envio para aprovação."

Identificação do cliente por CPF/CNPJ

CONFORME

O value object `CpfCnpj` normaliza a máscara e valida o dígito verificador de CPF (11) e CNPJ (14) antes de aceitar o documento — rejeita sequências repetidas e checksums inválidos.

```
domain/value_objects/cpf_cnpj.py
```

Cadastro do veículo (placa, marca, modelo, ano)

CONFORME

Entidade `Veiculo` exige marca/cliente válidos, valida formato de placa (antiga e Mercosul) via VO `Placa`, e limita o ano de fabricação a um intervalo plausível.

`domain/entities/veiculo.py`

`domain/value_objects/placa.py`

Inclusão de serviços solicitados

CONFORME

`OrdemServico.adicionar_item/remover_item` impedem duplicidade, quantidade ≤ 0 e valor negativo. Persistido em `ordem_servico_servicos`.

`domain/entities/ordem_servico.py:42-71`

Possibilidade de incluir peças e insumos na OS

LACUNA

Não existe caminho para anexar um insumo diretamente a uma OS. O único vínculo é transitivo — a "receita" fixa de um serviço (`ServicoInsumoModel`), exibida como somente-leitura no detalhe da OS. Não há coluna `insumo_id` em `ordem_servico_servicos`, nem endpoint, nem baixa de estoque associada a uma OS específica. Um mecânico não consegue registrar "usei 2 unidades deste filtro nesta ordem" fora da receita pré-cadastrada do serviço.

`infrastructure/database/models.py`

`presentation/api/routes/ordens_servico.py:451-453` (somente leitura)

▼ Sugestão de correção

Nova tabela de junção `ordem_servico_insumos` (mesmo padrão de `ordem_servico_servicos`), um método de domínio que reaproveita as mesmas validações de `adicionar_item`, e o use case chamando `InsumoRepository` para debitar o estoque no mesmo commit.

`infrastructure/database/models.py`

```
class OrdemServicoInsumoModel(Base):
    __tablename__ = "ordem_servico_insumos"

    ordem_servico_id = Column(Integer, ForeignKey("ordens_servico.id"),
    insumo_id = Column(Integer, ForeignKey("insumos.id"), primary_key=True)
    quantidade = Column(Integer, nullable=False)
    valor_unitario = Column(Numeric(10, 2), nullable=False)
```

`domain/entities/ordem_servico.py` – novo método, espelha `adicionar_item`

```
def adicionar_insumo(self, insumo_id: str, valor: float, quantidade: int):
    if not insumo_id:
        raise ValueError("ID do insumo é obrigatório")
    if quantidade <= 0:
        raise ValueError("Quantidade deve ser maior que zero")
    self.insumos_utilizados.append({
        "insumo_id": insumo_id, "valor": float(valor), "quantidade": quantidade
    })
    self._atualizar_timestamp()
```

No use case, chame `insumo.remover_estoque(quantidade)` (já existe e já valida saldo insuficiente) antes de persistir — reaproveita 100% da regra de estoque que já existe em `domain/entities/insumo.py`.

Orçamento gerado automaticamente a partir de serviços e peças

LACUNA

orcamento nunca é calculado — é sempre um número cru enviado pelo chamador da API, tanto na criação quanto em `enviar-approvacao` (campo obrigatório no request). Nenhum dos quatro pontos onde `orcamento` é atribuído soma `item["valor"] * item["quantidade"]`. Isso contraria literalmente o enunciado ("orçamento gerado automaticamente com base nos serviços e peças") e abre a porta para o cliente aprovar um valor diferente do que a OS realmente contém.

`domain/entities/ordem_servico.py:28`

`domain/entities/ordem_servico.py:93`

`application/commands/enviar_ordem_servico_para_aprovacao.py:35`

▼ Sugestão de correção

Adicionar uma propriedade calculada na entidade e usá-la como *default* em `enviar_para_aprovacao`, mantendo o parâmetro manual como override explícito (útil para descontos) em vez de removê-lo:

`domain/entities/ordem_servico.py`

```
@property
def orcamento_calculado(self) -> float:
    """Soma o valor dos itens já adicionados à OS."""
    return sum(item["valor"] * item["quantidade"] for item in self.itens)

def enviar_para_aprovacao(self, orcamento: float | None = None, observacao: str = ""):
    self._validar_transicao(StatusOrdemServico.AGUARDANDO_APROVACAO)
    orcamento_final = orcamento if orcamento is not None else self.orcamento_calculado
    if orcamento_final <= 0:
        raise ValueError("Orçamento deve ser um valor positivo")
    # ... resto do método permanece igual, usando orcamento_final
```

E no `EnviarOrdemServicoParaAprovacaoCommand`, tornar `orcamento` opcional (`float | None = None`) para que o caso de uso já sirva o fluxo 100% automático.

Envio do orçamento para aprovação do cliente

CONFORME

Use case dedicado (EnviarOrdemServicoParaAprovacaoUseCase) aplica a transição 'Em diagnóstico' → 'Aguardando aprovação' e persiste via o mesmo repositório usado no resto da aplicação.

```
application/commands/enviar_ordem_servico_para_aprovacao.py
```

B

Acompanhamento da Ordem de Serviço

"6 status; atualização automática por ações do sistema; cliente consulta o andamento via API."

Os 6 status, nomeados exatamente como no enunciado

CONFORME

StatusOrdemServico reproduz Recebida / Em diagnóstico / Aguardando aprovação / Em execução / Finalizada / Entregue palavra por palavra, com um grafo de transições explícito que já impede saltos inválidos (ex.: Recebida → Em execução direto).

```
domain/value_objects/status_ordem_servico.py
```

Atualização automática de status por ação do sistema

CONFORME

Cada transição tem um Command + UseCase dedicado (iniciar_diagnostico_ordem_servico, aprovar_orcamento_ordem_servico, etc.), seguindo o mesmo padrão do resto do app — a rota apenas monta o comando, o caso de uso decide e persiste.

```
application/commands/*_ordem_servico*.py
```

Cliente consulta o andamento via API

PARCIAL

O enunciado trata "cliente acompanha via app" e "APIs administrativas exigem JWT" como duas coisas separadas — sugerindo que a consulta do cliente não deveria depender do mesmo login administrativo. Hoje só existe um caminho: GET `/ordens-servico/{id}`, protegido pelo `get_current_user` administrativo. Um cliente final não tem como consultar sua própria OS sem uma conta de funcionário — não há token de cliente, nem endpoint público/escopado por CPF+placa.

`presentation/api/routes/ordens_servico.py:365-372`

▼ Sugestão de correção

Não duplicar a rota administrativa — expor uma consulta pública mínima, sem dados sensíveis de outros clientes, autenticada pela posse de dados que só o dono da OS teria (documento + placa), não por sessão administrativa:

```
@router.get("/consulta", response_model=OrdemServicoResponse)
async def consultar_publica(
    numero_os: int,
    cpf_cnpj: str,
    use_case: GetOrdemServicoDetalhadaUseCase = Depends(get_ordem_servi
):
    """Consulta pública: exige o número da OS + o CPF/CNPJ do cliente d
    detalhe = await use_case.execute(numero_os)
    if detalhe is None or detalhe.cliente is None:
        raise HTTPException(404, "Ordem de serviço não encontrada")
    if re.sub(r"\D", "", cpf_cnpj) != str(detalhe.cliente.cpf_cnpj):
        raise HTTPException(404, "Ordem de serviço não encontrada") #
    return resposta_resumida(detalhe) # menos dados que a rota admin
```

C

Gestão administrativa

"CRUD de clientes, veículos e serviços; CRUD (criar/ler/excluir) de peças com controle de estoque; listagem e detalhamento de OS; monitoramento do tempo médio de execução."

CRUD de clientes, veículos e serviços

CONFORME

Os três seguem o mesmo desenho: entidade rica valida no construtor, Command/UseCase por operação, repositório concreto por trás de uma interface ABC. Sem lacunas de negócio encontradas.

```
application/commands/{create,update,delete}_{cliente,veiculo,servico}.py
```

CRUD de peças/insumos com controle de estoque

CONFORME

O enunciado pede especificamente Create/Read/Delete — o repositório entrega isso e também Update, o que é estritamente mais permissivo, não uma violação. Estoque tem entrada, saída (valida saldo insuficiente) e ajuste absoluto, cada um com seu próprio caso de uso.

```
application/commands/{add,remove,adjust}_estoque.py
```

Listagem e detalhamento de OS

CONFORME

Listagem paginada com filtro por status, CPF/CNPJ do cliente e placa; detalhe agrega cliente, veículo, itens e os insumos de cada serviço num único DTO.

```
application/queries/list_ordens_servico.py
```

```
application/queries/get_ordem_servico_detalhada.py
```


Monitoramento do tempo médio de execução do serviço

LACUNA

Não implementado. Não há endpoint, caso de uso ou query relacionada a tempo médio, KPI, dashboard ou relatório em nenhuma das camadas. O único hit textual para "duração" é um campo livre e não estruturado (tempo_estimado) na descrição de um Serviço do catálogo — não é medição real de tempo de execução.

application/queries/ — nenhum resultado

▼ Sugestão de correção

O histórico de status já grava timestamp de cada transição (HistoricoOrdemServicoModel.data_status) — o material bruto para o cálculo já existe, só falta agregá-lo. Query dedicada, na mesma pasta das demais:

application/queries/get_tempo_medio_execucao.py

```
class GetTempoMedioExecucaoUseCase:
    """Tempo médio entre 'Em execução' e 'Finalizada', nas OS já finalizadas"""

    def __init__(self, ordem_servico_repository: OrdemServicoRepository):
        self.ordem_servico_repository = ordem_servico_repository

    async def execute(self, periodo_dias: int = 30) -> TempoMedioExecucao:
        ordens = await self.ordem_servico_repository.find_finalizadas_dentro_de(
            periodo_dias)
        duracoes = [o.duracao_execucao for o in ordens if o.duracao_execucao is not None]
        return TempoMedioExecucao(
            media_horas=sum(duracoes) / len(duracoes) if duracoes else None,
            amostra=len(duracoes),
        )
```

Expor como GET /ordens-servico/metricas/tempo-medio-execucao, protegido — é claramente um endpoint administrativo, não um dado de cliente final.

D

Segurança e qualidade

"Autenticação JWT nas APIs administrativas; validação de CPF/CNPJ e placa; testes automatizados."

JWT nas APIs administrativas

CRÍTICO

`get_current_user` existe e funciona — mas está preso em só **2 de ~34 endpoints** em toda a API (`GET /ordens-servico` e `GET /ordens-servico/{id}`). Todo o resto é acessível sem token: criar/editar/excluir clientes, veículos, serviços; criar/editar/excluir peças e *movimentar estoque* (entrada/saída/ajuste); criar uma OS; e as 8 transições de status (incluindo aprovar orçamento e entregar o veículo). Isso é o inverso do requisito — as leituras estão protegidas e as escritas administrativas, não.

```
presentation/api/routes/{clientes,insumos,servicos,veiculos}.py
```

- 0 endpoints protegidos `presentation/api/routes/ordens_servico.py`
- 2/11 protegidos

▼ Sugestão de correção

Em vez de colar `Depends(get_current_user)` em cada função (fácil de esquecer em endpoints novos), proteger no nível do `APIRouter` — um único ponto de verdade por domínio:

```
# presentation/api/routes/clientes.py
router = APIRouter(
    prefix="/clientes",
    tags=["Clientes"],
    dependencies=[Depends(get_current_user)], # aplica a TODAS as rotas
)
```

Repetir para `insumos.py`, `servicos.py`, `veiculos.py`. Em `ordens_servico.py`, mover a proteção do endpoint individual para o router também cobre a criação e as 8 transições de status de uma vez — reduzindo o diff e eliminando a chance de um novo endpoint nascer desprotegido por esquecimento.

Chave JWT: variável de ambiente inoperante

CRÍTICO

`.env.example` declara `JWT_SECRET_KEY / JWT_ALGORITHM / JWT_EXPIRE_MINUTES`, mas `Settings` (`pydantic-settings`) lê os nomes de campo `SECRET_KEY / ALGORITHM / ACCESS_TOKEN_EXPIRE_MINUTES`, sem `env_prefix` nem alias ligando um ao outro. Resultado: configurar `JWT_SECRET_KEY` no `.env` não tem efeito nenhum — em qualquer ambiente, inclusive produção, a aplicação sempre cai no valor hardcoded no repositório (`"42B38697D6C921058DDCFDD5ED5D89FAF0C671E3"`). Qualquer pessoa com acesso ao código-fonte consegue forjar um JWT administrativo válido — achado direto para o "relatório de vulnerabilidades" pedido no enunciado.

`.env.example` `infrastructure/config/config.py:3-6`

▼ Sugestão de correção

Alinhar os nomes com `env_prefix` e remover o fallback hardcoded — se a variável não estiver definida, a aplicação deve falhar ao subir, não usar um segredo público:

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="JWT_")

    SECRET_KEY: str # obrigatório - lê JWT_SECRET_KEY, sem default
    ALGORITHM: str = "HS256"
    EXPIRE_MINUTES: int = 60

settings = Settings()
```

E renomear os usos de `settings.ACCESS_TOKEN_EXPIRE_MINUTES` para `settings.EXPIRE_MINUTES` em `infrastructure/config/security.py`.

Validação de CPF/CNPJ e placa

CONFORME

Ambos com dígito verificador real (não só formato) nos VOs `CpfCnpj` e `Placa`, reaproveitados em toda entrada de cliente/veículo.

`domain/value_objects/{cpf_cnpj,placa}.py`

Testes unitários e de integração dos fluxos principais

CONFORME

485 testes, 97% de cobertura de linha+branch nos domínios críticos, separados em tests/unit e tests/integration (repositórios contra SQLite real + API ponta a ponta via TestClient) — acima do mínimo de 80% pedido.

tests/unit/

tests/integration/

E

Requisitos técnicos

"Monolito em camadas; Swagger; Dockerfile + docker-compose; README de configuração local."

Monolito, arquitetura em camadas / DDD

CONFORME

domain → application → infrastructure → presentation, dependência sempre apontando para dentro. Nenhuma camada externa vaza para o domínio.

APIs REST documentadas via Swagger

CONFORME

Gerado automaticamente pelo FastAPI a partir dos `response_model` e docstrings de cada rota, incluindo o esquema de segurança OAuth2 nos dois endpoints protegidos.

Dockerfile + docker-compose.yml

CONFORME

Ambos presentes na raiz; README documenta `docker compose up --build`.

README com configuração local

CONFORME

Cobre pré-requisitos, subida via Docker, logs, testes e cobertura. Não justifica explicitamente a escolha do PostgreSQL — item pedido no enunciado ("a escolha do banco é livre, mas é necessário justificá-la") e hoje ausente do texto.

`README.md`

2 achados críticos concentram o risco: superfície administrativa sem JWT e a chave JWT hardcoded inoperante — ambos no domínio D, ambos de baixo esforço para corrigir.